Muhammad Raza Nawaz

Machine learning Specialization by Andrew Ng

Supervised Machine learning: Regression and Classification Course: 01
week: 01

Machine learning: It is a science of getting computers to learn without being explicitly programmed.

Why is Machine learning so widely used today? Machine learning has grown up as subfield of Artificial Intelligence. we wanted to create intelligent systems, but it turned out that we can program machines to do most of the basic tasks, but for the more complex tasks we simply didn't know how to program, so the best practice was to let machine figure out itself to work on complex problems.

Artifical General Intelligence: A concept ~~in the~~ of developing systems as intelligent as humans meaning, they certainly do not require specific training for every possible scenario.

Machine learning (the formal def): Field of study that gives computers the ability to learn without being explicitly programmed - Arthur Samuel, 1959

- the more chances algorithm gets on training set, the better it performs.

Machine learning algorithms
- Supervised learning.
- Unsupervised learning.
- Recommender Systems.
- Reinforcement learning.

- In the course, will get the practical advice for applying these tools.

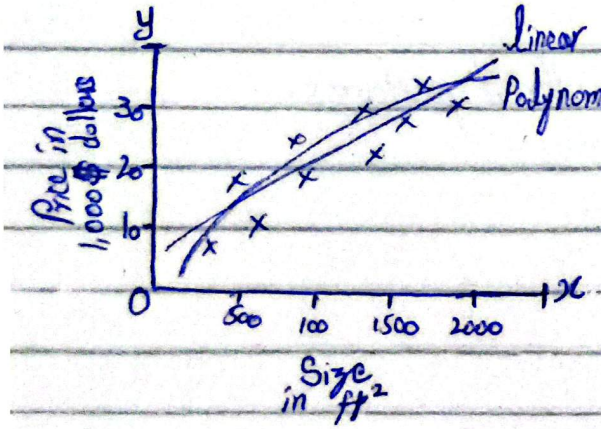
CamScanner

Supervised learning: refers to algorithms that learn input to output or x to y mappings.

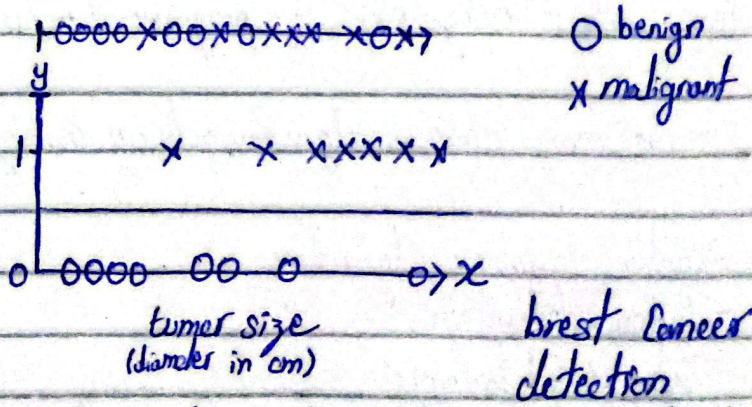
The key idea is that you give your algorithm examples to learn from, that include the right answers, whereby right answers means the correct label y for a given input x, by seeing the correct pairs of input feature x & desired output label y, the learning algorithm eventually learns to ~~just~~ take just the input alone, ~~without the~~ and give a reasonably accurate prediction.

- Regression: Predict a number from infinitely many possible numbers for a given input x. e.g: house price prediction.

- Classification: Predict a number/class from a small group of finite classes/categories e.g breast cancer detection.

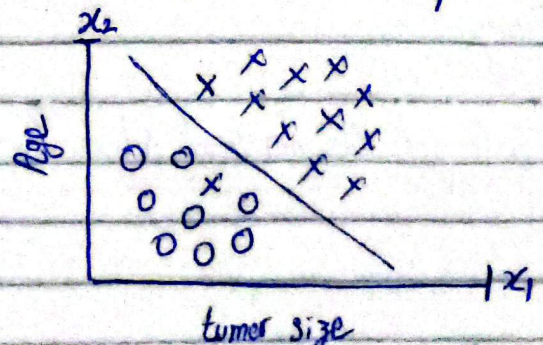


House price prediction
(Regression)



Classification with 1 input

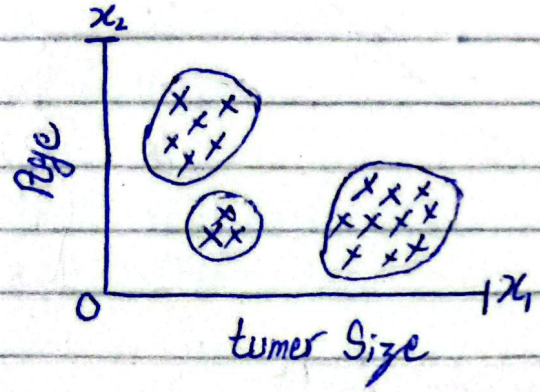
breast cancer detection
(Classification)



Classification with 2 inputs

Unsupervised learning: refers to algorithms which require data with just the input x and not the ~~outp~~ label y, the galgorithm itself finds interesting information hidden pattern or some structure in the data.

- Clustering: Group similar data points together.
- Anomaly detection: Find unusual datapoints or events.
- Dimensionality Reduction: Compress data with losing as little information as possible.



It is called unsupervised learning, because we are not supervising the algorithm to predict the output label y, instead we asked the algorithm to figure out all by yourself, whatis interesting in the data.

Clustering

Supervised learning (Regression)

Linear Regression: is a machine learning & statistical method that estimates the relationship between a dependent variable (scalar response) & one or more independent variables (explanatory variables) using a linear combination.

linear Combination: An expression formed by multiplying a set of variables (or vectors) by constants and adding the results together.

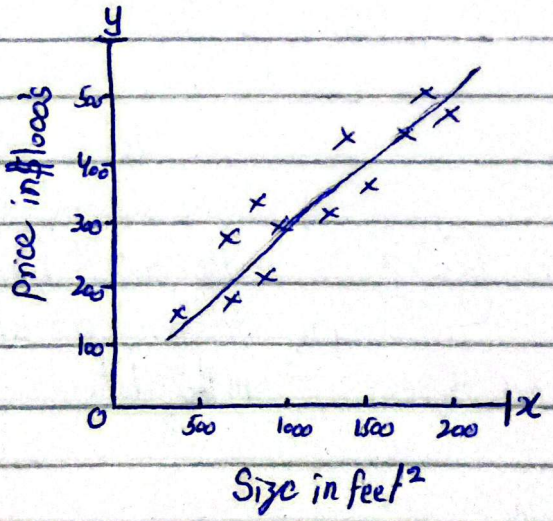
Univariate Vs Multivariate Linear Regression:
Univariate involves predicting a single target variable, Multivariate involves predicting multiple target variables.

Simple Vs Multiple Linear Regression:
A model with one independent variable is a simple linear Regression, while a model with multiple independent variable is a multiple linear Regression.

Example: House Price Prediction

Given: Size of house in ft^2 and price of house in $1000's dollars

Problem: Predict the price of house for every new x, size of house.
i.e the best fit line.



x	y
2104	400
1416	232
1534	315
852	178
...	...

Terminology

best fit line: A line that maps the relationship between two variables most efficiently.

dataset, training set
x, input variable / feature
y, output variable / target
m, no. of rows / training examples
(x, y), Single training example
$(x^{(i)}, y^{(i)})$, ith training example
$(x^{(2)}, y^{(2)})$, (1416, 232)

$\hat{y}$, estimation / prediction of y

training set → learning algorithm → $x \rightarrow f \rightarrow \hat{y}$

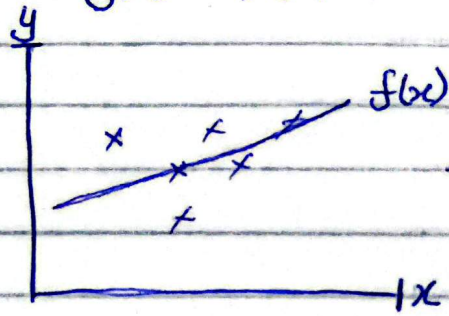
hypothesis / function / model

How to represent f?

for linear Regression,

$f_{w,b}(x) = wx + b$ "linear function"

$\hat{y}, f(x), f_{w,b}(x)$



$f(x) = wx + b$ "linear Regression"

linear, straight line.

w, b = weights, bias

w, b = slope, intercept

w, b = parameters

$wx + b$ = linear combination

best fit line

Univariate linear Regression

parameters: the variables, we can change during training in order to improve performance of the model.

---

with different values of w, b we will get different model, to ~~get~~ differentiate best model from all the other models, we use the cost function.

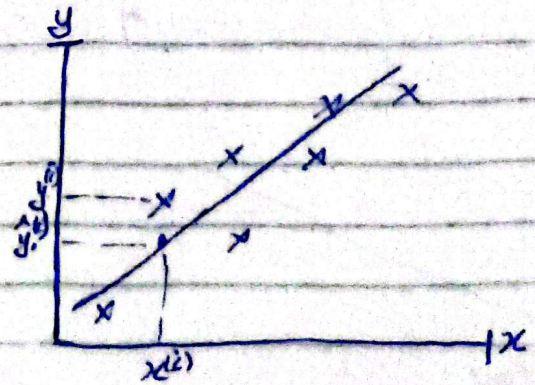
Cost function: is the measure of error between the models predicted values & the actual target values.

$$\text{error} = \hat{y}_i^i - y^i$$

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i^{(i)} - y^{(i)})^2$$

$$J(w,b) = 1/2m \sum_{i=1}^{m} (f_{w,b}(x_i^{(i)}) - y^{(i)})^2$$

"MSE"



model: $f_{w,b}(x) = wx + b$

parameters: w, b

cost function: $J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})^2$

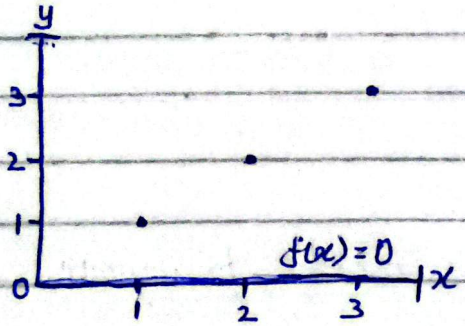
goal: $\underset{w,b}{\text{minimize}}\ J(w,b)$

Consider $(b = 0)$

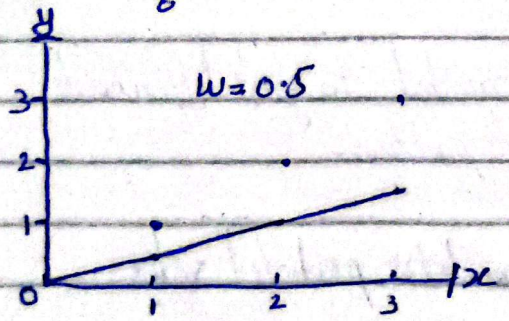
$f_w(x^{(i)}) = wx$

x	y
1	1
2	2
3	3

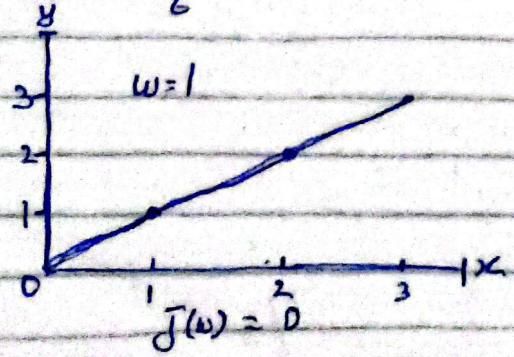
for $w = 0$

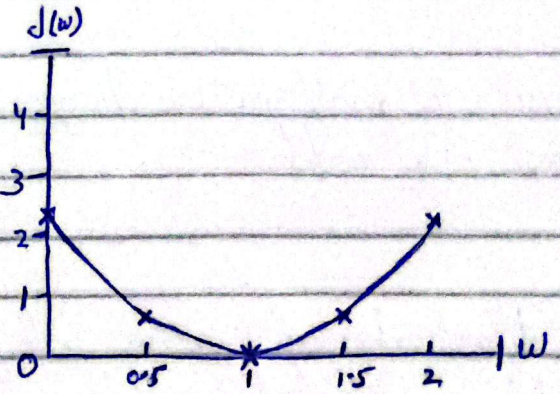


$J(w) = \frac{1}{6}(1^2 + 2^2 + 3^2) = 2.33$



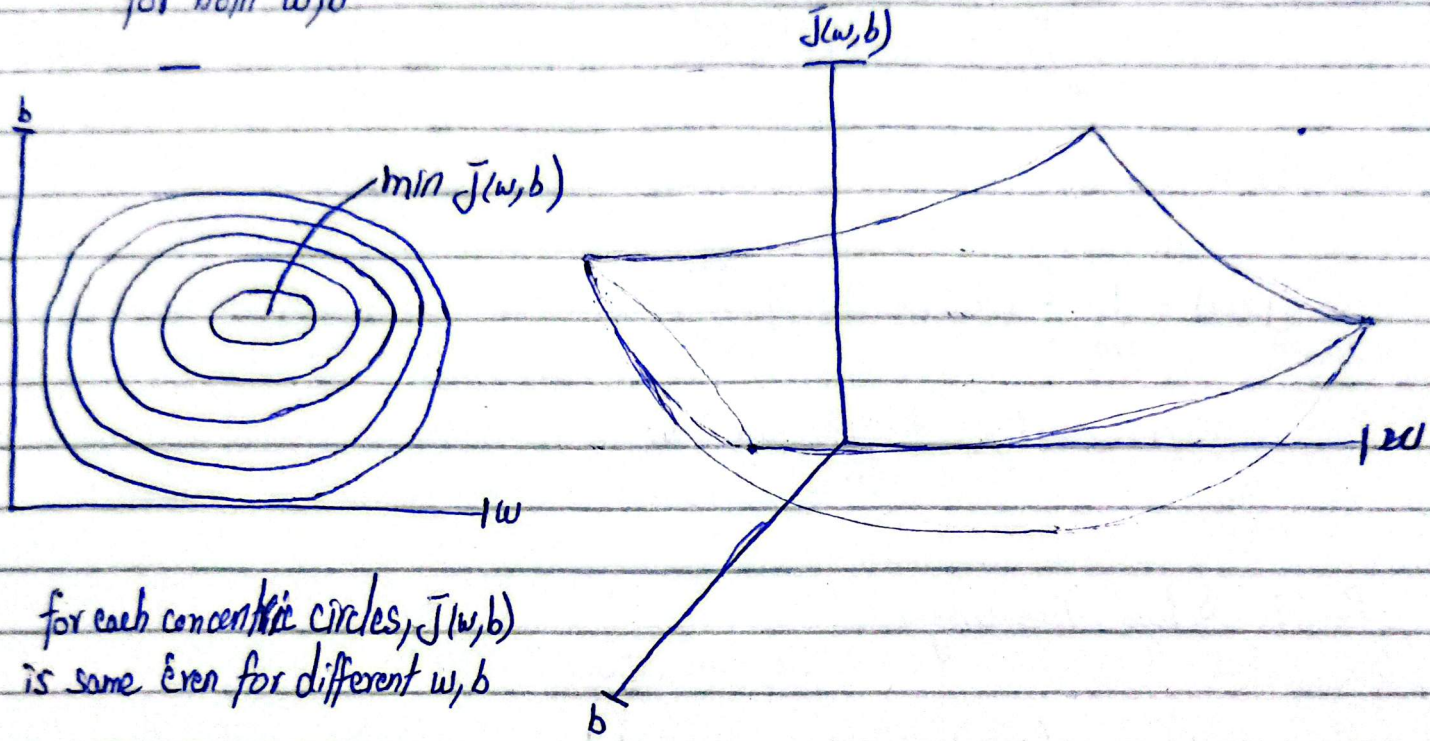
$J(w) = \frac{1}{6}(0.5^2 + 1^2 + 1.5^2) = 0.58$





vary w until, we get minimum J(w)
i.e this is the best model.

for both w,b



for each concentric circles, $J(w,b)$ is same even for different w,b

Now we got idea that $f(x)$ is a model but to get the best result of it, we need to change the w,b and find $\min J(w,b)$ using cost function, but exactly how much the change in w,b is required. To adress this problem, Gradient descent is used.

Gradient Descent: is the first order iterative optimization algorithm for finding the local minimum of the cost function. The idea is to take repeated steps in the opposite direction of a gradient of the function, because this is the direction of the steepest descent, conversely stepping in the direction of gradient will lead to local maxima. then the algorithm will be known as Gradient Ascent.

$$w_{new} = w_{old} - \alpha \frac{\partial J(w,b)}{\partial w_{old}}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J(w,b)}{\partial b_{old}}$$

α = learning rate

$\frac{\partial J(w,b)}{\partial w}$ = partial derivative

Intuition

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

consider b = constant

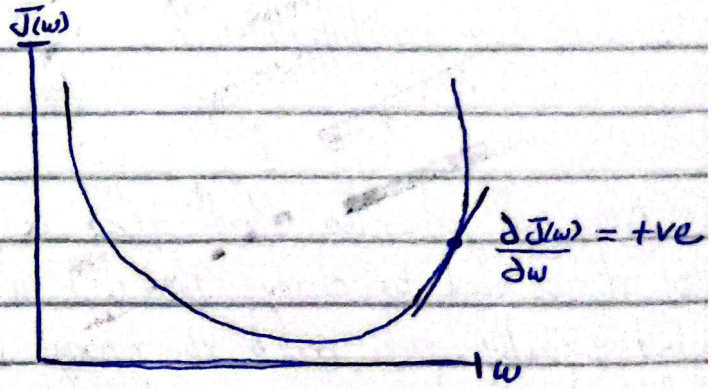
$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w}(x^{(i)}) - y^{(i)})^2$$

$\frac{\partial J(w)}{\partial w}$ = slope of $\underset{\text{cost}}{J(w)}$ w.r.t w

Case 1: Slope Increasing

$w_{new} = w_{old} - \alpha(+ve)$

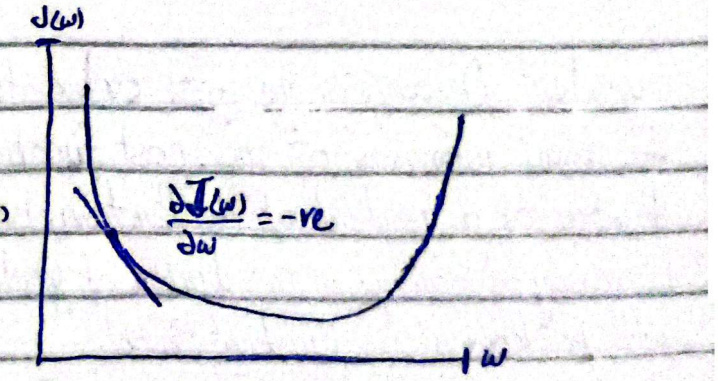
w_{new} = decreasing "moving to minimum"



Case 2: slope decreasing

$w_{new} = w_{old} - \alpha(-ve)$

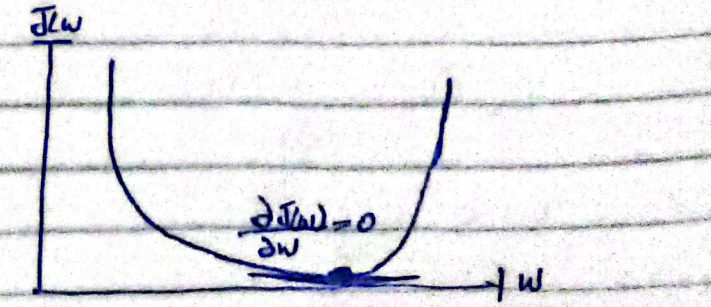
w_{new} = Increasing "moving to minimum"



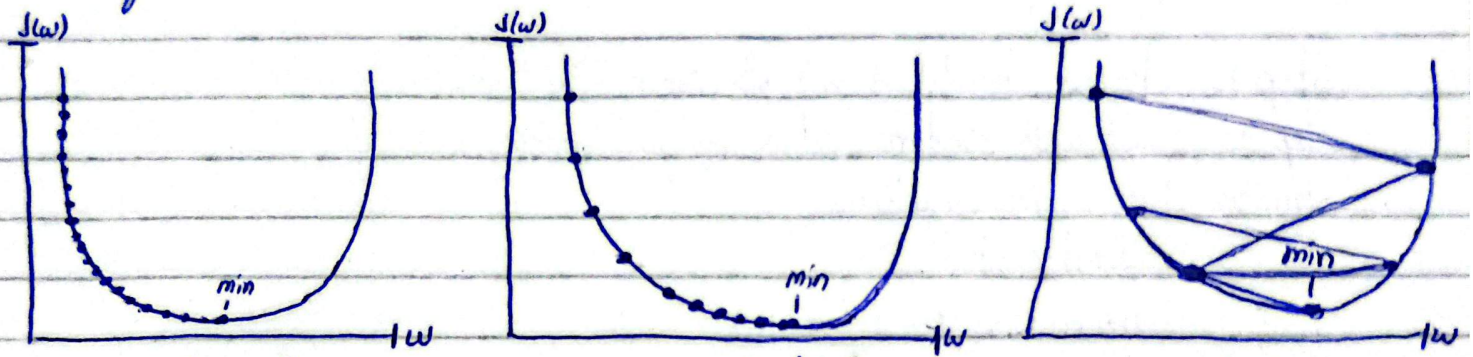
Case 3: slope constant

$w_{new} = w_{old} - \alpha(0)$

$w_{new} = w_{old}$



learning rate (α)



α = to small
converge, much more time

α = moderate
converge, with perfect time

α = to high
may diverge

derivation

$$\frac{\partial J(w,b)}{\partial w} = \frac{\partial}{\partial w}\left(\frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2\right) \qquad \because f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

$$= \frac{1}{2m} \cdot 2\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right) \cdot x^{(i)}$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right) \cdot x^{(i)}$$

$$\frac{\partial J(w,b)}{\partial b} = \frac{\partial}{\partial b}\left(\frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2\right)$$

$$= \frac{1}{2m} \cdot 2\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)} - y^{(i)})\right)$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)} - y^{(i)})\right)$$

Linear Regression

step 1: choose model $f(x)$ & cost function.
step 2: start with random values of parameters i.e w, b.
step 3: for i in range(m):
find cost

$$w_{new} = w_{old} - \alpha \frac{\partial J(w,b)}{\partial w_{old}}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J(w,b)}{\partial b_{old}}$$

recap:

model: $f_{w,b}(x^{(i)}) = wx + b$

cost function: $J(w,b) = \frac{1}{2m} \cdot \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$

Gradient descent: $w_{new} = w_{old} - \alpha \frac{\partial J(w,b)}{\partial w}$

$$b_{new} = b_{old} - \alpha \frac{\partial J(w,b)}{\partial b}$$

Some other important concepts:

Convex function: A function having only one local minimum, which is also the global the global minimum of function, ex: MSE.

Batch Gradient descent: A type of gradient descent which ~~uses~~ calculates the total error over all training examples to perform an update a singk update of parameters.

Week:02

Multiple Linear Regression: is a machine learning method used to predict an output variable y by using two or more independent explanatory features $(x_1, x_2 \dots x_n)$.

Size in ft^2	No. of bedrooms	No. of floor	Age of home in yrs	Price in 1000's dollars
x_1	x_2	x_3	x_4	y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

x_j = j^{th} feature

$x_2 = [5\ 3\ 3\ 2 \dots]$

n = no. of features/columns

$x^{(i)}$ = i^{th} training example

$x^{(2)} = [1416\ 3\ 2\ 40]$

$x_j^{(i)}$ = value of specific j^{th} feature in specific i^{th} training example

$x_3^{(2)} = 3$

$$f_{\vec{w},b}(\vec{x}) = \underbrace{\vec{w} \cdot \vec{x}}_{\text{dot product}} + b$$

$$= w_1 \cdot x_1 + w_2 x_2 + w_3 \cdot x_3 + w_4 \cdot x_4 + b$$

In general, $f = w_1 \cdot x_1 + w_2 x_2 + \cdots + w_n x_n + b$

Vectorization: replacing slow loops of codes with fast array or matrix operations.

without vectorization

```
for i in range(n):
    f = f + w[i] * x[i]
```

with vectorization

$$\vec{w} \cdot \vec{x} = [w_1\ w_2\ w_3\ w_4 \cdots] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

fast for large datasets

In Gradient descent "Vectorization"

$$w = w - \alpha \frac{\partial J(w,b)}{\partial w}$$

$$[w_1\ w_2\ w_3\ \cdots] - \alpha \cdot \left[\frac{\partial J}{\partial w_1}\ \frac{\partial J}{\partial w_2}\ \frac{\partial J}{\partial w_3}\ \cdots\right]$$

Cost function

$$J(w_1, \cdots, w_n, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w_1,\cdots,w_n b}(x^{(i)}) - y^{(i)}\right)^2$$

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)^2$$

Gradient descent

$$w_{j_{new}} = w_{j_{old}} - \alpha \frac{\partial J(w_1, \cdots w_n, b)}{\partial w_{j_{old}}}$$

$$w_j = w_{j_{old}} - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_{j_{old}}} \qquad \frac{\partial J(\vec{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J(\vec{w}, b)}{\partial b_{old}} \qquad \frac{\partial J(\vec{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$$

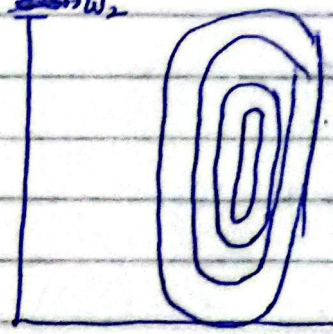
feature Scaling

$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + b$

x_1 size x_2 bedrooms

Size in ft²	bedrooms	Price
1007	5	
887	3	
900	2	

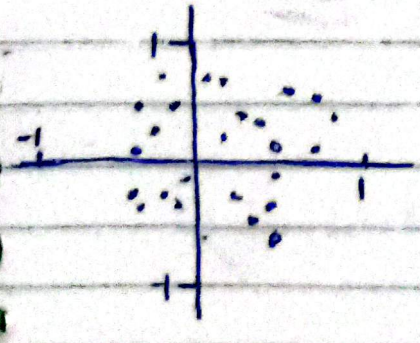
As w_1 is related to a feature with bigger values, therefore the model will observe more rapid changes in x_1 and will give more importance to w_1, will ignoring w_2 but both the features were equally important to predict y. also the contour plot of the dataset will be elongated in one direction causing gradient descent to oscillate while finding the minima. To address this problem feature scaling is used.



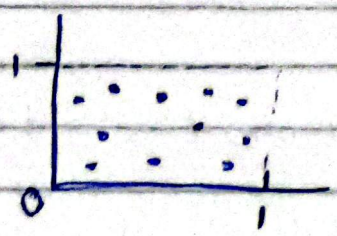
Simple feature scaling: $\frac{x_i}{x_{max}}$

Normalization (Minmax scaler): $\frac{x_i - \mu}{x_{max} - x_{min}}$

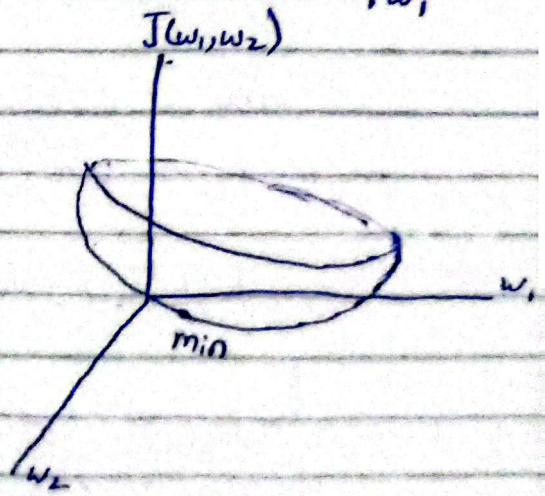
Z-score Normalization "Standardization" (Standard scaler): $\frac{x_i - \mu}{\sigma}$



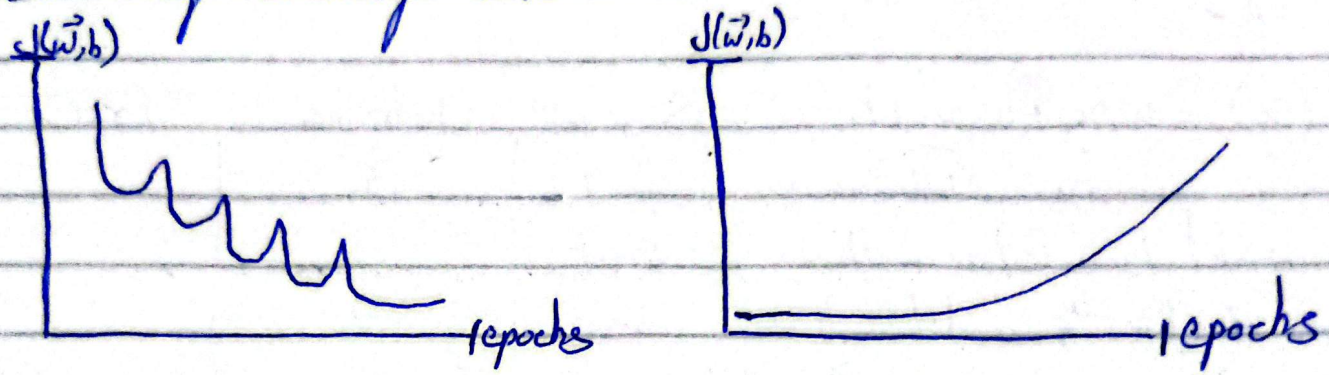
Standardization
zer-centric
[-1, 1] range 'Not compulsory'



Normalization
[0,1] range 'forced in b/w'
~~mean~~ centric range centric



Choosing learning rate



↳ too large or bug in code

↳ choosing strategy

0.001 → 0.003 → 0.01 → 0.03 → 0.1 → 0.3 → 1

(3x) (3x) (3x) (3x) (3x) (3x)

feature engineering

Suppose, $f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + b$

(x_1: frontage, x_2: depth)

area = frontage × depth

$x_3 = x_1x_2$ (new feature)

$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$

Def: Using intuition to design new features, by transforming or combining original features.

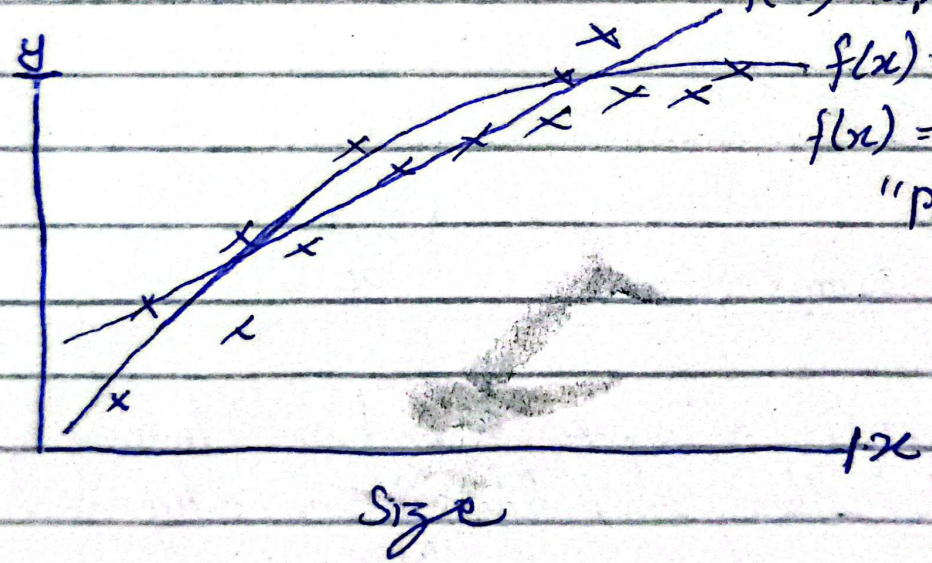
Polynomial Regression: is a type of regression used in machine learning that models the relation ship between x and y using polynomial equations. It is useful when the data follows a curved pattern instead of a straight line.

$f(x) = w_1 x + b$ 'linear Regression

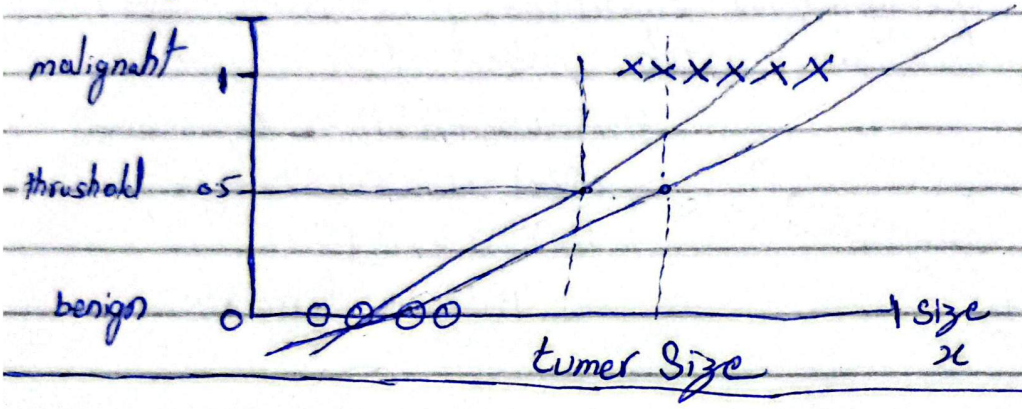
$f(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$

$f(x) = w_1 x + w_2 \sqrt{x} + b$

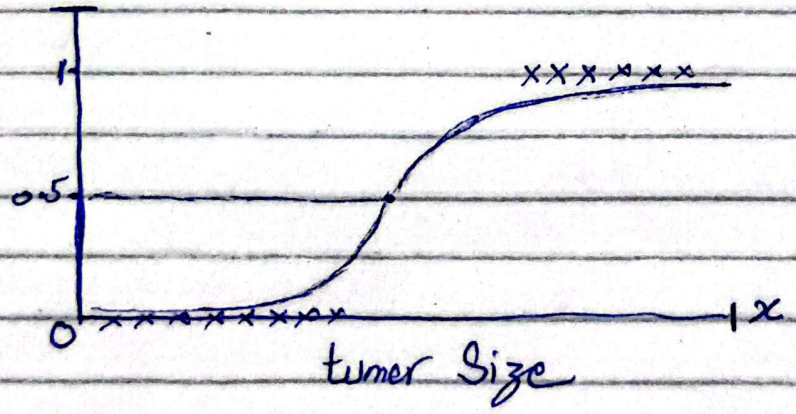
"Polynomical regression



Why Linear Regression can't be applied to Classification?



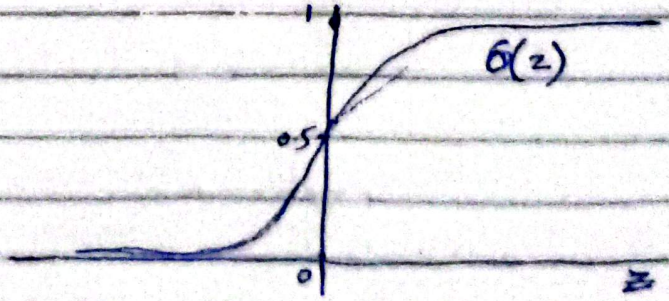
Logistic Regression: is a supervised machine learning & statistical model used to predict the probability of a categorical (discrete) outcome.

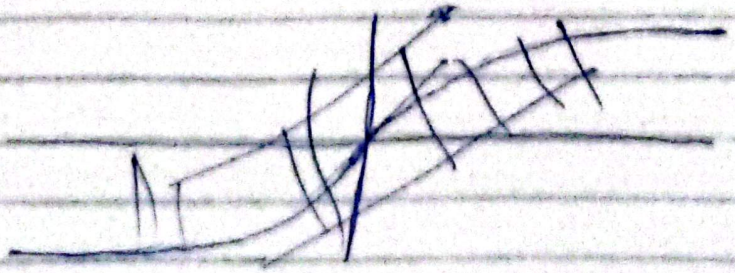


$$f_{w,b}(x) = g(w \cdot x + b)$$

g = sigmoid

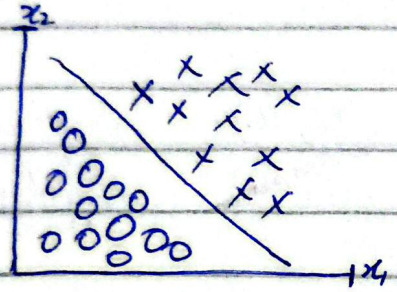
$$g(wx+b) = \frac{1}{1+e^{-(wx+b)}}$$





Decision boundary: is a hyperplane, line, or curve in a feature space that partitions data points into different predicted classes

Linear decision boundaries



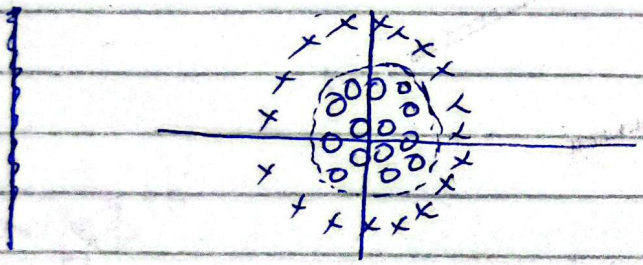
$f_{\vec{w},b}(x) = g(z) = g(w_1x_1 + w_2x_2 + b)$

$z = \vec{w} \cdot x + b$

$z = x_1 + x_2 - 3 = 0 \quad w_1 = 1, w_2 = 1, b = -3$

$\Rightarrow x_1 + x_2 = 3$

Non-linear decision boundaries



$w_1 = 1, w_2 = 1, b = -1$

$z = w_1x_1^2 + w_2x_2^2 + b = 0$

$\Rightarrow x_1^2 + x_2^2 - 1 = 0$

$x_1^2 + x_2^2 = 1$

We can get even more complex non-linear decision boundaries if

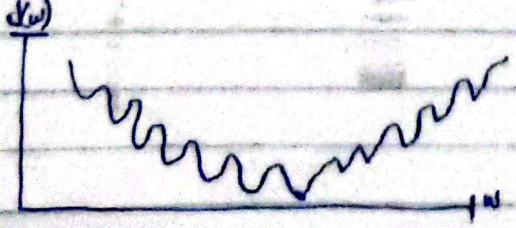
$g(z) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + b)$

Cost function

Why not MSE for logistic regression?

$MSE \Rightarrow J(\vec{w},b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)^2$

where, $f(\vec{w},b)(x^{(i)}) = 6(\vec{w}\vec{x} + b)$



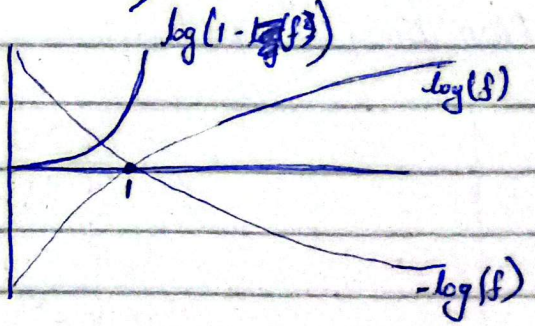
Non-Convex function: function having more than one local minimas.

loss function: is a mathemical formula to calculate the error between the actual & predicted values over a single training example.

loss function of logistic regression:

$$loss = l(f_{\vec{w},b}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{w,b}(x^{(i)})) & \text{if } y^{(i)}=1 \\ -\log(1-f_{w,b}(x^{(i)})) & \text{if } y^{(i)}=0 \end{cases}$$

$$l = -y^{(i)}\log(f_{w,b}(x^{(i)})) - (1-y^{(i)})\log(1-f_{w,b}(x^{(i)}))$$



Cost function

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}\left(-y^{(i)}\log(f_{w,b}(x^{(i)})) - (1-y^{(i)})\left(\log(1-f_{w,b}(x^{(i)}))\right)\right.$$

Convex function

Gradient descent:

$$w_{new} = w_{old} - \alpha \frac{\partial J(\vec{w},b)}{\partial w_{j\,old}} \qquad \frac{\partial J(\vec{w},b)}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

$$b_{new} = b_{old} - \alpha \frac{\partial J(\vec{w},b)}{\partial b_{old}} \qquad \frac{\partial J(\vec{w},b)}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$$

Recap

model : $f_{\vec{w},b}(\vec{x}) = g(\vec{w}\cdot\vec{x}+b)$ $\qquad g(z) = \frac{1}{1+e^{-z}}$

Cost function : $\frac{1}{m}\sum_{i=1}^{m}\left(-y^{(i)}\log\left(f_{\vec{w},b}(x^{(i)})\right) - (1-y^{(i)})\log\left(1-f_{\vec{w},b}(x^{(i)})\right)\right)$

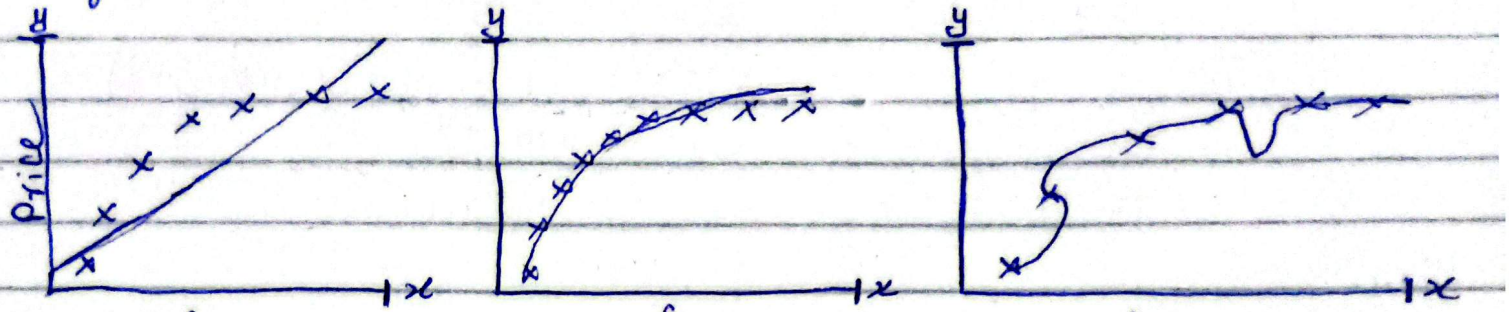
Gradient descent:

$$w_{j\,new} = w_{j\,old} - \alpha \frac{\partial J(\vec{w},b)}{\partial w_{j\,old}} \;;\; \frac{\partial J(\vec{w},b)}{\partial w_{j\,old}} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

$$b_{j\,new} = b_{j\,old} - \alpha \frac{\partial J(\vec{w},b)}{\partial b_{j\,old}} \;;\; \frac{\partial J(\vec{w},b)}{\partial b_{j\,old}} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$$

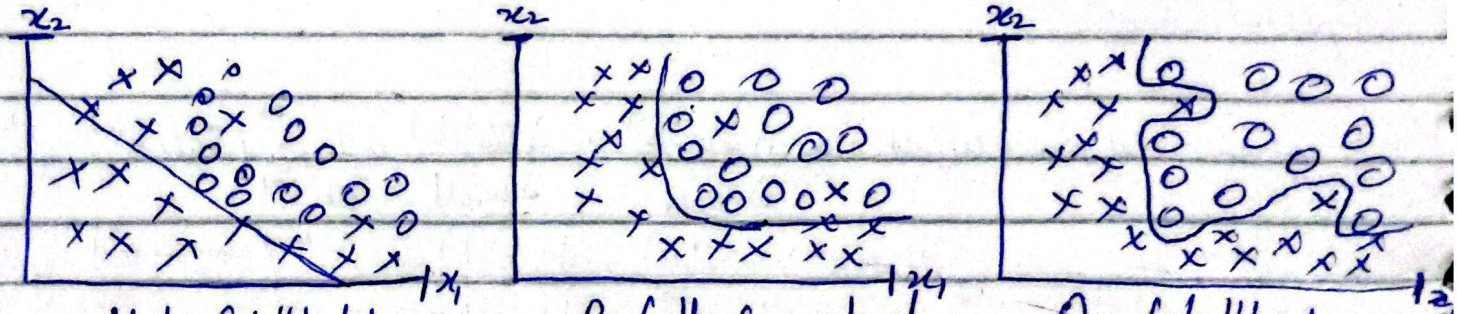
The problem of overfitting

Regression



Underfit "high bias" | Perfectly generalized | Overfit "high variance"

Classification



Under fit "high bias"
$z = w_1x_1 + w_2x_2 + b$

Perfectly Generalized
$z = w_1x_1 + w_2x_2 + w_3x_1x_2 + b$

Overfit "high variance"
$z = w_1x_1 + w_2x_1^2 + w_3x_2 + w_4x_2^2 + w_5x_1x_2 + b$

Addressing Overfitting

- Collect more training data.
- Use fewer features. 'feature selection'
- Regularization 'reducing weights values'

Cost function with regularization

$$J(\vec{w},b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right) + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

$\lambda \gg 0$ regularization effect is stronger

Gradient descent

$$w_j = w_j - \alpha \frac{\partial J(\vec{w},b)}{\partial w_j}$$

$$= w_j - \alpha \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)x_j^{(i)} - \alpha\lambda/m\, w_j$$

$$= w_j\left(1 - \alpha\frac{\lambda}{m}\right) - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$$

Regularized linear Regression

Cost function: $J(\vec{w},b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right) + \lambda/2m\sum_{j=1}^{n} w_j^2$

Gradient descent $\Rightarrow w_j = w_j(1 - \alpha\lambda/m) - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$

Regularized logistic regression

Cost function: $J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left(-y\log\left(f_{\vec{w},b}(x^{(i)})\right) - (1-y^{(i)})\log\left(1 - f_{\vec{w},b}(x^{(i)})\right)\right) + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$

Gradient descent: $w_j = w_j(1 - \alpha\lambda/m) - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(x^{(i)}) - y^{(i)}\right)$

CamScanner